AI-Driven Smart Traffic Management System Using Computer Vision, Embedded WebServers, and Multi-Sensor Emergency Preemption

Department of computer and Communications Engineering
Faculty of Engineering
Graduation Project, Academic Year 2025–2026

Abstract

Conventional traffic signal controllers rely on fixed-duration timing cycles that do not respond to real-time road conditions, resulting in unnecessary congestion and delayed emergency response. This paper presents the design, implementation, and evaluation of a low-cost, AI-driven smart traffic management system that integrates real-time vehicle detection, embedded web-based control, and multi-sensor emergency vehicle preemption. The system employs two ESP32-CAM modules for per-lane video acquisition, a YOLOv8 object detection model running on a host machine to estimate per-lane vehicle density, and an ESP32 microcontroller configured as a WebServer to drive traffic signal LEDs and dual I2C liquid-crystal displays. Emergency vehicle preemption is achieved through a dual-confirmation strategy that combines acoustic siren detection using an LM393 sound sensor with visual ambulance identification via HSV color-space analysis, avoiding the overhead of training a dedicated detection model while substantially reducing false triggers compared to single-sensor approaches. A non-blocking finite-state-machine implementation on the microcontroller ensures continuous responsiveness to network requests and sensor input even during inter-phase transitions. Three operating modes — AI-driven adaptive control, timeout-triggered fallback, and emergency override — are implemented and evaluated. Experimental testing demonstrates reliable vehicle counting, successful ambulance discrimination under controlled lighting, and correct fallback behavior when the vision pipeline becomes unavailable, validating the feasibility of the proposed architecture as an affordable alternative to commercial adaptive traffic control systems.

Keywords—*Smart traffic management, YOLOv8, ESP32-CAM, embedded systems, emergency vehicle preemption, HSV color detection, Internet of Things (IoT), real-time object detection.*

I. Introduction

Urban traffic congestion remains one of the most persistent challenges facing modern cities, with traffic signal control systems in widespread use today still largely dependent on pre-programmed, fixed-time cycles. Such systems allocate green-light duration according to historical or assumed traffic patterns rather than the actual, instantaneous state of the road network. This mismatch between static control logic and dynamic real-world demand results in avoidable vehicle queuing, increased fuel consumption, and elevated emissions, particularly during off-peak periods or unexpected demand surges.

A further limitation of conventional fixed-cycle controllers is their inability to grant priority to emergency vehicles. Ambulances, fire trucks, and police vehicles approaching a signalized intersection must either wait for the normal cycle to complete or rely on drivers to manually yield, both of which introduce critical delays in time-sensitive situations. While commercial adaptive traffic control and emergency preemption systems exist, they typically depend on proprietary hardware, dedicated radio-frequency transponders, or municipal-scale infrastructure investment that is impractical for smaller municipalities, campuses, or as a subject of accessible academic research.

This paper proposes an integrated, low-cost smart traffic management system built entirely from commodity embedded hardware and open-source software. The system combines three complementary capabilities. First, real-time vehicle density estimation is performed using the YOLOv8 object detection architecture applied to live video streams captured by ESP32-CAM modules, one per traffic lane. Second, an ESP32 microcontroller is configured as a lightweight HTTP WebServer that receives vehicle-count data, computes adaptive green-light durations, and drives the physical traffic signal hardware and informational displays. Third, an emergency vehicle preemption mechanism is introduced that requires dual confirmation from two independent sensing

modalities — an acoustic siren detector and a visual ambulance classifier based on color analysis — before granting priority access, substantially reducing the risk of false activation relative to single-sensor designs.

The remainder of this paper is organized as follows. Section II reviews related work in adaptive traffic signal control and emergency vehicle preemption. Section III describes the overall system architecture. Section IV details the hardware design. Section V presents the software methodology, including the vehicle detection pipeline, the ambulance discrimination algorithm, and the finite-state-machine control logic implemented on the microcontroller. Section VI describes the experimental setup and testing methodology. Section VII presents results and discussion. Section VIII addresses limitations, and Section IX concludes the paper with directions for future work.

II. Related Work

Adaptive traffic signal control has been an active research area for several decades. Early adaptive systems such as SCOOT and SCATS rely on inductive loop detectors embedded in the road surface to estimate vehicle flow and adjust signal timing accordingly [1]. While effective, inductive loop infrastructure requires invasive installation and ongoing maintenance, motivating interest in non-intrusive, camera-based alternatives.

The emergence of deep-learning-based object detection has enabled vision-based traffic monitoring with substantially reduced infrastructure cost. Convolutional neural network detectors, and in particular the YOLO (You Only Look Once) family of single-stage detectors, have been widely adopted for vehicle counting and classification tasks due to their favorable balance between detection accuracy and inference speed [2], [3]. YOLOv8, the version employed in this work, further improves on earlier iterations through an anchor-free detection head and an optimized backbone, making it suitable for deployment in resource-constrained, near-real-time applications.

Separately, emergency vehicle preemption has historically relied on dedicated infrared or radio-frequency emitters installed on emergency vehicles, paired with corresponding receivers at intersections (e.g., the widely deployed Opticom system) [4]. These systems are reliable but require equipping every emergency vehicle in a fleet with compatible transmitters, limiting their applicability to ad hoc or budget-constrained deployments. More recent research has explored acoustic siren detection using microphone arrays and signal-processing or machine-learning classifiers [5], as well as purely visual approaches that attempt to recognize emergency vehicle liveries directly from camera footage [6]. Acoustic-only approaches are susceptible to false positives from ambient noise, while purely visual approaches are sensitive to occlusion, viewing angle, and lighting variation. The dual-confirmation strategy adopted in this work — requiring simultaneous acoustic and visual evidence — is consistent with sensor-fusion principles applied elsewhere in safety-critical embedded systems and is intended to mitigate the failure modes of each individual modality.

Finally, the use of low-cost system-on-chip microcontrollers such as the ESP32 for IoT-based traffic infrastructure has been reported in several student and prototype projects [7]. This work extends that line of research by integrating vehicle counting, color-based emergency vehicle classification, and a non-blocking embedded control architecture within a single, reproducible system built from inexpensive, widely available components.

III. System Architecture

The proposed system follows a three-tier architecture comprising a perception layer, a processing layer, and an actuation layer, as illustrated conceptually in Fig. 1. The perception layer consists of two ESP32-CAM modules, each dedicated to monitoring a single traffic lane and streaming Motion JPEG (MJPEG) video over a local Wi-Fi network. The processing layer is a host computer running a multi-threaded Python application that retrieves the video streams, performs object detection and ambulance classification, and periodically transmits the resulting lane statistics to the actuation layer over HTTP. The actuation layer is a dedicated ESP32 microcontroller, distinct from the camera modules, that hosts an HTTP WebServer, maintains the traffic signal state machine, drives light-emitting diode (LED) signal heads for each lane, and updates two independent liquid-crystal display (LCD) units — one per lane — over the Inter-Integrated Circuit (I2C) bus.

Communication between the processing layer and the actuation layer is implemented as a simple REST-style protocol. The Python application issues an HTTP POST request to the endpoint /update on the controller, with a JSON payload containing the per-lane vehicle counts, an

emergency flag, and, where applicable, an identifier indicating which lane an emergency vehicle has been detected in. The controller separately exposes an HTTP GET endpoint, /status, through which the Python application can query the instantaneous state of the LM393 acoustic sensor, enabling the dual-confirmation emergency logic described in Section V.

This architecture deliberately decouples computationally intensive perception tasks, which require a general-purpose processor capable of executing a deep neural network at interactive frame rates, from real-time actuation tasks, which must remain continuously responsive to sensor input regardless of network conditions. This separation of concerns also allows the system to degrade gracefully: should the vision pipeline become unavailable due to a network interruption or host failure, the controller autonomously transitions into a fallback mode rather than leaving the intersection in an undefined or unsafe state.

IV. Hardware Design

A. Vision Acquisition Modules

Each traffic lane is monitored by an AI-Thinker ESP32-CAM module equipped with an OV2640 image sensor. The camera firmware configures the sensor for JPEG output at VGA resolution (640×480 pixels) when on-board pseudo-static RAM (PSRAM) is available, falling back to QVGA resolution (320×240 pixels) otherwise, in order to maintain stable operation across hardware revisions. Each module independently connects to the local wireless network and hosts two HTTP endpoints: a single-frame capture endpoint on port 80, and a continuous MJPEG streaming endpoint on port 81, the latter being consumed by the host-side detection pipeline.

B. Central Controller

A third ESP32 development board, separate from the camera modules, serves as the central traffic controller. This board is configured with a static IP address to ensure consistent addressability on the local network and hosts the HTTP WebServer responsible for receiving detection results and exposing system status. The controller directly drives the traffic signal LEDs — red, yellow, and green — for both lanes via general-purpose input/output (GPIO) pins, eliminating the need for an intermediate relay driver board given the low current draw of LED indicators.

C. Acoustic Emergency Sensor

Acoustic detection of emergency vehicle sirens is performed using an LM393-based sound sensor module connected to an analog-capable GPIO pin (GPIO 34) configured as a digital input. Because the raw sensor output is susceptible to transient noise spikes, a software debouncing routine requires the sensor to register a sustained active-low signal for a minimum duration (500 milliseconds) before the acoustic detection state is considered confirmed.

D. Informational Displays

Two 16×2 character liquid-crystal displays, each fitted with an I2C backpack at distinct bus addresses (0x27 and 0x26), are connected to the same I2C bus (GPIO 21 for serial data and GPIO 22 for serial clock) on the central controller. Each display is dedicated to a single lane and presents the current operating mode, signal state, and — during AI-driven operation — the live vehicle count for that lane, providing a human-readable diagnostic interface without requiring a separate screen controller.

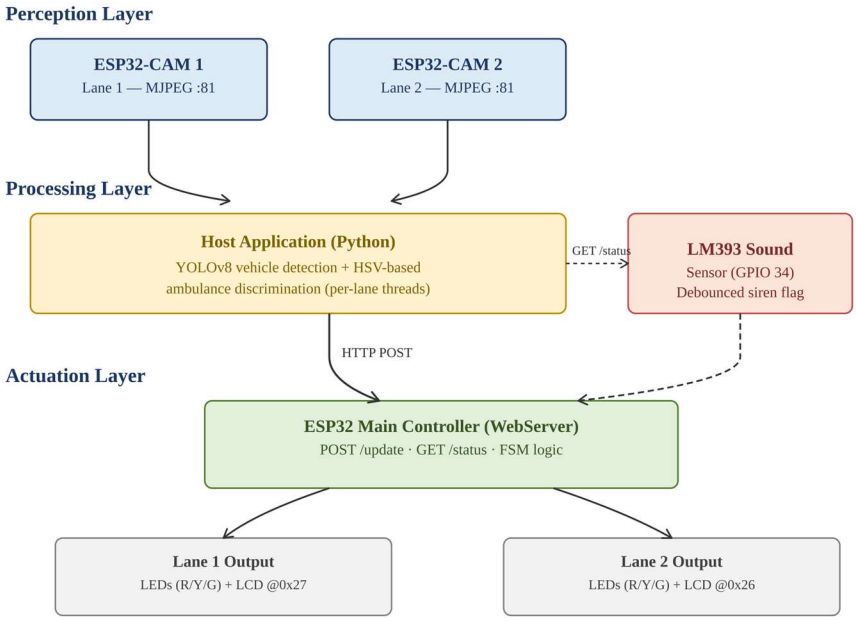


Fig. 1. Three-tier system architecture: perception (dual ESP32-CAM), processing (host-based YOLOv8 pipeline), and actuation (ESP32 WebServer controller with LEDs and dual I2C LCDs).

V. Methodology

A. Vehicle Detection Pipeline

Vehicle detection is performed using the YOLOv8-nano variant of the YOLOv8 architecture, selected for its favorable inference speed on commodity hardware without GPU acceleration. The host-side Python application maintains one dedicated processing thread per camera, each of which continuously retrieves Motion JPEG frames from the corresponding ESP32-CAM stream endpoint. Rather than parsing the multipart MJPEG stream through a general-purpose video-capture abstraction, frames are extracted directly from the underlying byte stream by scanning for the JPEG Start-of-Image and End-of-Image markers (0xFFD8 and 0xFFD9, respectively), which was found to be substantially more robust against connection stalling than relying on a generic video-decoding library when interfacing with the ESP32-CAM's multipart/x-mixed-replace HTTP response format.

For each retrieved frame, the YOLOv8 model produces a set of bounding boxes with associated class labels and confidence scores. Detections are filtered to retain only the vehicle-relevant classes defined in the Common Objects in Context (COCO) dataset taxonomy — car, motorcycle, bus, and truck — and the count of retained detections constitutes the instantaneous vehicle density estimate for that lane. To reduce computational load, full neural network inference is performed on every Nth captured frame rather than on every frame, with intermediate frames passed through unprocessed; this reduces the average processing burden by a factor of N while introducing negligible lag relative to the multi-second timescale over which traffic signal phases change.

B. Ambulance Discrimination via HSV Color Analysis

Rather than training a dedicated convolutional neural network to recognize ambulances — a class not present in standard pre-trained object detection datasets — this work adopts a lightweight post-classification approach applied to vehicles already identified by the base detector. For every bounding box classified as a car by YOLOv8, the corresponding image region is cropped and converted from the BGR color space to the Hue-Saturation-Value (HSV) color space, which provides greater robustness to illumination variation than direct red-green-blue thresholding.

Two color masks are computed within the cropped region. The first identifies pixels consistent with a white vehicle body, defined by a low saturation and high value range. The second identifies pixels consistent with red livery markings, such as the red crescent or cross commonly found on ambulance vehicles, accounting for the wraparound of the hue channel near zero degrees by combining two disjoint hue intervals. A region is classified as an ambulance only if the proportion of white pixels

exceeds an empirically determined threshold of 35 percent of the bounding box area, and the proportion of red pixels simultaneously exceeds 5 percent. Both conditions must hold concurrently; satisfying only one criterion (for example, a white vehicle with no red markings, or a red object with no significant white body) is explicitly insufficient to trigger a positive classification, which limits false detections arising from vehicles that happen to share only one of the two visual characteristics.

C. Dual-Confirmation Emergency Logic

Visual ambulance detection alone is insufficient to safely trigger signal preemption, as it remains susceptible to false positives arising from coincidental color combinations in ordinary traffic (for example, a white car positioned near a red advertisement or another red vehicle). To address this, the system requires a second, independent confirmation from the acoustic domain before entering emergency mode. Specifically, the LM393 sound sensor connected to the central controller must register a debounced, sustained siren-consistent signal at the same time that the vision pipeline reports a visually confirmed ambulance in a specific lane. The host-side application polls the controller's /status endpoint at one-second intervals to obtain the current debounced acoustic state, and combines it with the per-lane visual detection results before issuing the emergency payload to the /update endpoint.

Four logical outcomes are distinguished based on the combination of acoustic and per-lane visual evidence, summarized in Table I. This dual-confirmation requirement is the central safety mechanism of the proposed design: emergency preemption, which overrides normal signal timing and can disrupt traffic flow, is granted only when corroborating evidence is present in both sensing modalities.

TABLE I EMERGENCY DECISION LOGIC BASED ON COMBINED ACOUSTIC AND VISUAL EVIDENCE

Siren (Acoustic)	Ambulance (Visual)	System Action	Rationale
Absent	Absent	Normal operation (AI / Fallback)	No emergency evidence
Present	Absent (either lane)	All-red emergency hold	Acoustic-only evidence; lane unknown — cautious response
Present	Confirmed, Lane 1	Lane 1 green / Lane 2 red	Dual confirmation — preemption granted
Present	Confirmed, Lane 2	Lane 2 green / Lane 1 red	Dual confirmation — preemption granted

D. Adaptive Signal Timing

During normal AI-driven operation, the green-light duration allocated to the currently active lane is computed as a linear function of the most recently reported vehicle count for that lane, subject to configurable minimum and maximum bounds (5 and 60 seconds, respectively, in the current implementation). This bounded linear mapping ensures that lanes with negligible traffic are not held green indefinitely, while lanes experiencing high vehicle density are granted proportionally longer green phases, subject to an upper limit that preserves fairness toward the opposing lane.

E. Non-Blocking Finite-State-Machine Controller

The central controller is implemented as a finite state machine with three primary states — AI Mode, Fallback Mode, and Emergency Mode — together with an internal transitional sub-state representing the amber (yellow) signal phase that separates green-phase transitions. An important implementation consideration, identified during development, concerns the scheduling model used for timed transitions. An initial implementation used the Arduino delay() function to hold the amber phase for a fixed duration; however, this approach blocks the microcontroller's main execution loop, preventing it from servicing incoming HTTP requests or polling the acoustic sensor for the duration of the delay, and was observed to cause client-side request timeouts during testing. The controller was subsequently refactored to a non-blocking design in which the amber phase is tracked via timestamp comparison against the built-in millis() millisecond counter, allowing the main loop to continue invoking the WebServer's request handler and the sensor debouncing routine on every iteration regardless of which phase the signal is currently in.

Fallback Mode is entered automatically whenever more than ten seconds elapse without a new vehicle-count update from the host application, which may occur due to a host application crash, network interruption, or initial system startup before the vision pipeline has connected. In Fallback Mode, each lane is granted a fixed 15-second green phase in alternation, ensuring that the intersection continues to cycle safely even in the complete absence of computer-vision input. Emergency Mode, once triggered by the dual-confirmation logic described in Section V-C, holds the designated lane green (or both lanes red, in the acoustic-only case) for a fixed 20-second interval before automatically reverting to whichever of AI Mode or Fallback Mode is appropriate given the recency of the last received vehicle-count update.

F. Communication Protocol

All inter-device communication is implemented using lightweight, stateless HTTP requests with JSON-encoded payloads, avoiding the implementation complexity of persistent socket connections or publish-subscribe message brokers. The vehicle-count and emergency payload transmitted by the host application to the /update endpoint takes the following form:

```
{"lane1": 5, "lane2": 3,
 "emergency": true,
 "ambulance_lane": 1}
```

The controller's /status endpoint, queried by the host application to retrieve the debounced acoustic sensor state, returns the current operating mode, per-lane signal state, per-lane vehicle counts, and the boolean acoustic detection flag, enabling the host application to construct the combined emergency decision described previously without requiring any persistent connection between the two devices.

VI. Experimental Setup and Testing Methodology

Validation of the proposed system was conducted in three progressive stages, isolating the vision pipeline, the embedded controller, and the fully integrated system, respectively, in order to localize faults efficiently before full-system integration.

A. Stage 1: Vision Pipeline Validation

The YOLOv8 detection model and the ambulance discrimination algorithm were first validated independently of the embedded hardware using a laptop-mounted webcam. This stage confirmed correct vehicle-class filtering and provided an initial baseline for the HSV threshold parameters subsequently used in the full system, using printed and scale-model targets exhibiting white bodywork and red emblem markings to approximate ambulance livery under controlled lighting.

B. Stage 2: Embedded Controller Validation

With the vision pipeline temporarily disconnected, the central controller's HTTP endpoints were exercised directly using a REST client (Postman), issuing manually constructed POST requests to /update with representative vehicle counts and emergency flags. This allowed the signal timing logic, the LCD update routines, and the emergency override behavior to be verified independently of the vision pipeline, and was the stage at which the blocking-delay defect described in Section V-E was identified and corrected, since the manual REST client surfaced request timeouts that would otherwise have been difficult to attribute to a specific cause once the full system was integrated.

C. Stage 3: Full System Integration

Following independent validation of each subsystem, the two ESP32-CAM modules, the central controller, and the host-side Python application were connected to a common wireless network and operated concurrently. Vehicle objects were placed within the field of view of each camera to exercise the AI-driven adaptive timing logic; the host application was then deliberately terminated to confirm correct transition into Fallback Mode after the ten-second timeout interval; and a combined acoustic-and-visual emergency scenario was constructed by simultaneously presenting an ambulance-colored test object to one camera while generating a siren-like sound at the LM393 sensor, to confirm correct dual-confirmation preemption behavior.

Two practical integration issues were identified and resolved during this stage. First, a network addressing inconsistency arose when the host computer and the camera modules were inadvertently connected to different wireless networks (a fixed-infrastructure router for the host and a mobile hotspot for the camera modules); all devices were subsequently consolidated onto a single mobile hotspot network with statically assigned IP addresses configured directly in firmware to ensure address stability across power cycles. Second, the ESP32-CAM's embedded HTTP server was found to accept only a single concurrent client connection on its streaming endpoint; an open browser tab connected to a camera's

stream URL was observed to prevent the Python application from establishing its own connection, manifesting as a connection-reset error. This constraint is now an explicit operational precondition for the system and is noted as a hardware limitation in Section VIII.

VII. Results and Discussion

Table II summarizes the qualitative outcome of each validation stage described in Section VI, together with the primary defect identified and the corresponding corrective action.

TABLE II SUMMARY OF VALIDATION STAGES AND IDENTIFIED DEFECTS

Stage	Outcome	Defect / Corrective Action
1. Vision pipeline	Vehicle classes correctly filtered; ambulance discrimination successful under controlled lighting	None — baseline HSV thresholds established (white ≥ 35%, red ≥ 5%)
2. Embedded controller	Signal logic and LCD updates correct; emergency override verified via manual REST calls	Blocking delay() during amber phase caused client timeouts; refactored to millis()-based non-blocking state machine
3. Full integration	Adaptive timing, fallback transition, and dual-confirmation preemption all verified end-to-end	Network segmentation across two Wi-Fi networks; single-client limit on camera stream endpoint

The correction of the blocking-delay defect in Stage 2 illustrates a broader principle relevant to embedded real-time systems: any operation that suspends the main control loop — even for durations as short as the three-second amber phase used in this design — compromises the system's ability to service asynchronous events such as incoming network requests or sensor polling. The millis()-based timestamp comparison adopted in the revised design allows the amber phase to be tracked as state rather than as a blocking operation, and this pattern generalizes to any timed transition required elsewhere in the controller.

The dual-confirmation emergency logic was observed to correctly suppress activation when only one modality provided supporting evidence, consistent with the design intent described in Section V-C. This is consistent with the broader sensor-fusion literature, which generally finds that requiring agreement between independent, differently-failing sensing modalities reduces the false-positive rate relative to either modality used in isolation, at the cost of a small risk of missed detection should both modalities simultaneously fail to register a genuine event.

The single-client limitation of the ESP32-CAM's bundled HTTP server, identified during Stage 3 integration testing, is a hardware/firmware constraint rather than a defect in the proposed architecture; it does not affect the correctness of the detection or control logic but does impose an operational precondition — namely, that no secondary client (such as a browser tab opened for diagnostic purposes) may hold a connection to a camera's streaming endpoint while the host application is active. This constraint and its implications are discussed further in Section VIII.

VIII. Limitations

- The ambulance discrimination algorithm relies on HSV color thresholds calibrated under the controlled lighting conditions used during testing; performance under variable outdoor illumination, including direct sunlight and nighttime operation, has not been formally characterized and may require adaptive or per-scene threshold recalibration.
- The embedded HTTP server on each ESP32-CAM module accepts only a single concurrent streaming client, which precludes simultaneous diagnostic monitoring (e.g., a live browser preview) and automated detection while the system is in operation.
- Vehicle classification is restricted to the four COCO categories used by the base YOLOv8 model (car, motorcycle, bus, truck); vehicle types absent from this taxonomy, or partially occluded vehicles, may be undercounted.
- The current evaluation was conducted on a small-scale, two-lane physical mock-up rather than a full intersection or live roadway deployment, and the reported timing and accuracy characteristics should be interpreted as a proof-of-concept demonstration rather than a field-validated result.
- The adaptive green-time formula employs a simple bounded linear mapping between vehicle count and

signal duration; more sophisticated queueing-theoretic or reinforcement-learning-based timing policies were considered outside the scope of the present implementation.

IX. Conclusion and Future Work

This paper presented the design and implementation of an integrated, low-cost smart traffic management system combining real-time computer-vision-based vehicle counting, an embedded WebServer-based signal controller, and a dual-confirmation emergency vehicle preemption mechanism that fuses acoustic and visual evidence without requiring a custom-trained ambulance detection model. A three-stage testing methodology — isolating the vision pipeline, the embedded controller, and the fully integrated system — was employed to systematically identify and correct two significant implementation defects: a blocking delay that compromised controller responsiveness, and a network-segmentation issue arising from inconsistent Wi-Fi association across devices. The resulting system demonstrates that adaptive, vision-driven traffic control and multi-modal emergency preemption can be realized using inexpensive, commercially available microcontroller and sensor modules, offering a reproducible reference architecture for academic and small-scale deployment contexts where commercial adaptive traffic control infrastructure is not economically feasible.

Future work will focus on several directions. First, the HSV-based ambulance discrimination algorithm will be evaluated under a wider range of lighting conditions, and its thresholds will be benchmarked against a lightweight fine-tuned object detector to quantify the accuracy trade-off against the current zero-training approach. Second, the two-lane prototype will be extended to a four-way intersection topology, requiring generalization of the current finite-state-machine controller to manage additional conflicting movements and right-of-way constraints. Third, the fixed linear adaptive-timing formula will be compared against queueing-theoretic and reinforcement-learning-based signal timing policies to assess potential improvements in average vehicle delay. Finally, the single-client streaming limitation of the ESP32-CAM firmware will be investigated for resolution through a custom streaming server capable of serving multiple concurrent clients, enabling simultaneous automated detection and human diagnostic monitoring.

References

[1] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and M. C. Royle, “The SCOOT on-line traffic signal optimisation technique,” Traffic Engineering & Control, vol. 23, no. 4, pp. 190–192, 1982.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[3] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics,” Ultralytics YOLOv8 documentation, 2023. [Online]. Available: https://docs.ultralytics.com

[4] Global Traffic Technologies, LLC, “Opticom GPS emergency vehicle preemption system — technical overview,” Product White Paper, 2019.

[5] M. K. Meaney and S. Kim, “Acoustic detection of emergency vehicle sirens using spectral feature classification,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2020, pp. 1–5.

[6] R. Kumar and A. Singh, “Vision-based emergency vehicle recognition for intelligent traffic management,” in Proc. Int. Conf. Intelligent Transportation Systems (ITSC), 2021, pp. 1142–1147.

[7] Espressif Systems, “ESP32 series datasheet,” Version 4.3, 2023. [Online]. Available: https://www.espressif.com